

Industrial Internship Report on "SERVANTA – Multi-client website offering client services"

**Prepared by
ANGEL MARIYA T J**

Executive Summary

This report presents the Industrial Internship completed under Upskill Campus and The IoT Academy, in collaboration with UniConverge Technologies Pvt. Ltd. (UCT). The internship involved developing Servanta, a role-based multi-tenant service marketplace connecting customers with merchants offering various services.

Servanta was developed using React.js with TypeScript for the frontend, Node.js with Express.js for the backend, and PostgreSQL as the database. Git and GitHub were used for version control. The system includes authentication, role-based access control, service management, booking, payment, search, reviews, notifications, and administrative features.

The project was developed through requirement analysis, system design, database development, frontend-backend integration, testing, and security implementation. JWT authentication, Helmet, CORS, rate limiting, and input validation were implemented to improve application security. The payment functionality uses a Razorpay test/simulated workflow without processing real monetary transactions.

By the end of the internship, Servanta had become a functional marketplace supporting Administrator, Merchant, and Customer roles. The project provided valuable practical experience in full-stack development, database management, API integration, authentication, payment workflows, security, testing, and software engineering practices.

TABLE OF CONTENTS

1	Preface.....	4
2	Introduction.....	6
2.1	About UniConverge Technologies Pvt Ltd.....	6
2.2	About upskill Campus (USC).....	6
2.3	The IoT Academy.....	6
2.4	Objectives of this Internship Program.....	7
2.5	Glossary.....	7
3	Problem Statement.....	9
4	Existing and Proposed Solution.....	11
4.1	Existing Solutions and Their Limitations.....	11
4.2	Proposed Solution – SERVANTA.....	11
5	Proposed Design / Model.....	15
5.1	System Architecture.....	15
5.2	High Level Diagram.....	16
5.3	Low Level Diagram.....	17
5.4	Entity-Relationship Diagram.....	18
5.5	Data Flow Diagram.....	19
5.6	Flowchart.....	20
5.7	Database Design.....	21
6	Project Modules.....	19
6.1	Authentication Module.....	23
6.2	Administrator Module.....	23
6.3	Merchant Module.....	23
6.4	Customer Module.....	24
6.5	Service Management Module.....	25
6.6	Booking Module.....	26
6.7	Payment Module.....	26
6.8	Review Module.....	27
6.9	Notification Module.....	28

6.10	Dashboard Module.....	28
7	Implementation.....	29
7.1	Folder Structure and Django Apps.....	30
7.2	Templates, Bootstrap and Static Files.....	30
7.3	Environment Variables and Configuration.....	30
7.4	Version Control with Git and GitHub.....	30
7.5	Database Configuration and Requirements.....	30
7.6	Project Organisation and Code Reusability.....	31
8	Testing and Performance Evaluation.....	32
8.1	Test Plan / Test Cases.....	32
8.2	Test Procedure.....	33
8.3	Performance Evaluation.....	34
9	GitHub Repository.....	35
9.1	Repository Structure.....	35
9.2	Version Control and Commit History.....	35
9.3	README.....	35
9.4	Repository Link.....	35
10	Screenshots.....	36
11	My Learnings.....	42
12	Future Work Scope.....	44
13	Conclusion.....	45
14	References.....	46
15	Appendix.....	47

1 PREFACE

This report documents the four-week Industrial Internship completed with **UniConverge Technologies Pvt. Ltd. (UCT)**, delivered through the **Upskill Campus and The IoT Academy** programme. As part of the internship, I designed and developed **Servanta**, a multi-tenant service marketplace, progressing from initial requirement analysis and system planning to the development, testing, security enhancement, and completion of a functional web application.

An industrial internship of this nature provides an experience that classroom learning alone cannot fully offer. It places a practical problem in front of a student within a defined timeframe and requires the application of technical knowledge, problem-solving skills, technology selection, and implementation decisions in a manner similar to a real software development environment. For a final-year Computer Science Engineering student, this practical exposure provides valuable insight into how software systems are planned, developed, tested, secured, and maintained.

Full-stack web development particularly benefits from hands-on implementation. Concepts such as client-server communication, RESTful APIs, database relationships, authentication and authorization, role-based access control, responsive user interfaces, and frontend-backend integration become much easier to understand when they are implemented and tested as part of a complete application. Working on Servanta provided practical experience in connecting these individual concepts into a unified software system.

The internship followed a progressive development approach, beginning with requirement analysis and project planning, followed by environment setup, database and system design, frontend interface development, backend API implementation, authentication, and role-based functionality. The subsequent stages focused on implementing the core marketplace features, including service management, service discovery, booking management, reviews, payment workflow, and administrative functionality. The final stages involved integration, testing, security improvements, optimisation, debugging, and preparation of the completed application for demonstration and submission.

Working on Servanta throughout the internship provided a clearer understanding of the responsibilities involved in full-stack application development beyond individual tutorials or isolated programming exercises. The project required designing a PostgreSQL database capable of supporting different user roles and marketplace relationships, developing RESTful backend APIs using **Node.js and Express.js**, creating an interactive frontend using **React.js and TypeScript**, and integrating the different components into a reliable application. It also provided practical experience with authentication, payment workflows, API communication, error handling, security practices, and version control using Git and GitHub.

I would like to express my sincere gratitude to the mentors and team at **UniConverge Technologies Pvt. Ltd.** for their guidance and support throughout the internship. I am also thankful to **Upskill Campus and The IoT Academy** for organising and facilitating the programme. I extend my sincere gratitude to the faculty and management of **Vidya Academy of Science and Technology** for their continuous encouragement and support during the internship period.

To juniors and peers preparing for similar industrial internships, my suggestion is to give equal importance to planning, implementation, testing, and documentation. Proper requirement analysis and system design at the beginning of Servanta helped make the later development stages more structured and manageable. The experience demonstrated that successful software development is not only about writing code, but also about understanding requirements, making appropriate technical decisions, testing thoroughly, addressing security concerns, and continuously improving the system.

2 Introduction

2.1 About UniConverge Technologies Pvt. Ltd.

UniConverge Technologies Pvt. Ltd. (UCT) is a technology-driven organisation working in the field of digital transformation and industrial technology solutions. The company focuses on developing technology solutions that improve operational efficiency, sustainability, and business outcomes. UCT works with modern technologies across areas such as the **Internet of Things (IoT), cybersecurity, cloud computing, machine learning, communication technologies, and full-stack software development.**

UCT's technology portfolio includes solutions for IoT-based device connectivity, industrial monitoring, analytics, smart manufacturing, asset management, predictive maintenance, and other digital transformation applications. The organisation also provides opportunities for students and aspiring developers to gain practical exposure through industry-oriented internship and project programmes.

This internship was carried out under UCT's **Full Stack Development** track. As part of the programme, the assigned project was **Servanta**, a multi-tenant service marketplace developed to connect customers with merchants and enable them to discover, manage, and book different services through a centralized web platform.

2.2 About Upskill Campus (USC)

Upskill Campus (USC), in association with The IoT Academy and UniConverge Technologies, facilitated the execution of the industrial internship programme. The platform focuses on providing learners with industry-oriented training and career development opportunities through practical learning experiences.

Through the internship programme, students are provided with opportunities to work on technology-based projects, gain practical development experience, and understand professional software development practices. The programme helped bridge the gap between academic knowledge and the technical skills required in an industrial environment.

2.3 The IoT Academy

The IoT Academy is the EdTech division of UniConverge Technologies that provides technology-oriented education and professional certification programmes in collaboration with reputed academic and industry institutions.

The academy focuses on developing practical skills in emerging technologies and provides learners with industry-relevant training and project-based learning opportunities. As part of this internship programme, The IoT Academy contributed to

providing a structured learning environment and facilitating the industrial project experience.

2.4 Objectives of the Internship Program

- Gain practical experience in a professional software development environment.
- Apply academic knowledge to the development of a real-world full-stack web application.
- Understand the complete software development lifecycle, from requirement analysis to testing and deployment preparation.
- Develop practical skills in **React.js, TypeScript, Node.js, Express.js, PostgreSQL, REST APIs, and Git/GitHub**.
- Understand frontend-backend integration and database-driven application development.
- Implement authentication, authorization, role-based access control, and security mechanisms in a web application.
- Develop practical understanding of marketplace features such as service management, booking, reviews, and payment workflows.
- Improve problem-solving, debugging, testing, communication, teamwork, and professional development skills.
- Gain industry-oriented experience that can contribute to future career opportunities in full-stack web development.
- Successfully design and implement **Servanta**, a functional multi-tenant service marketplace supporting Administrator, Merchant, and Customer roles.

2.5 Glossary

Term	Acronym
Create, Read, Update, Delete	CRUD
Object-Relational Mapping	ORM
Model-View-Template	MVT
Application Programming Interface	API
User Interface / User Experience	UI/UX

Structured Query Language	SQL
HyperText Markup Language	HTML
Cascading Style Sheets	CSS
Representational State Transfer	REST

3 PROBLEM STATEMENT

A multi-tenant service marketplace is an online platform that connects independent service providers, referred to as **merchants**, with customers looking for different services. Unlike a traditional single-vendor website, a multi-tenant marketplace allows multiple merchants to operate independently within a single platform, while customers can discover and interact with services offered by different providers. **Servanta** was developed as a role-based service marketplace following this model, providing a centralized platform for service discovery, booking, payment, and service management.

Many independent service providers and small businesses face difficulties in establishing and maintaining an effective online presence. Developing a separate website and digital management system for every service provider can be expensive and technically demanding. At the same time, customers increasingly prefer to search for services, compare available options, make bookings, and manage their interactions through a single online platform.

The problem addressed by the Servanta project can be summarized as follows:

- Independent service providers need an affordable and centralized platform to showcase, manage, and promote their services online.
- Customers may find it difficult to discover and compare services from multiple independent providers through a single platform.
- Traditional single-vendor systems do not provide sufficient support for multiple merchants to independently manage their services and bookings.
- The absence of an integrated platform for service discovery, booking, payment, and review management can make the service-booking process inefficient.
- Manual handling of service availability, bookings, customer information, and communication can lead to delays and errors.
- Merchants require dedicated functionality to manage their services, pricing, availability, bookings, and customer interactions.
- Platform administrators require centralized tools to manage users, merchants, services, categories, bookings, and overall platform activity.
- A secure role-based system is required to ensure that customers, merchants, and administrators can access only the features and information relevant to their respective roles.

These challenges motivated the development of **Servanta** as a secure, role-based, multi-tenant service marketplace. The platform enables merchants to independently manage their services and bookings, allows customers to discover and book services through a

centralized interface, and provides administrators with centralized control and monitoring capabilities.

The system also incorporates features such as **authentication and authorization, service search and filtering, booking management, reviews, payment workflow, role-based access control, and security mechanisms** to provide a complete and reliable marketplace experience. The remainder of this report explains how these requirements were analysed, designed, implemented, integrated, tested, and refined throughout the internship.

4 EXISTING AND PROPOSED SOLUTION

4.1 Existing Solutions and Their Limitations

Several types of online platforms currently address different aspects of the service marketplace problem. Large service marketplaces provide customers with access to multiple service providers, but their broad scale and competitive environment can make it difficult for smaller and independent merchants to effectively manage and promote their services. Category-specific platforms provide focused solutions for particular services, but they may not offer a flexible structure for supporting multiple service categories and different types of merchants within a single system.

Many small and independent service providers also continue to depend on traditional methods such as telephone calls, messaging applications, social media, and manual scheduling for managing their services and customer bookings. Although these approaches are simple to use, they can become inefficient as the number of customers and bookings increases. They may also result in missed requests, scheduling conflicts, difficulty maintaining booking records, and limited access to structured customer feedback.

Existing approaches may therefore lack a unified combination of **multi-merchant service management, service discovery, booking, payment processing, reviews, role-based access, and centralized administration**. These limitations created the need for a platform that could bring these functions together while allowing each merchant to independently manage their own services.

4.2 Proposed Solution – Servanta

Servanta is a full-stack, multi-tenant service marketplace developed to connect customers with independent merchants through a centralized web platform. The system is designed around three primary roles: **Administrator, Merchant, and Customer**. Each role has dedicated functionality and access permissions according to its responsibilities.

The application uses **React.js with TypeScript** for the frontend, **Node.js with Express.js** for the backend, and **PostgreSQL** for data management. RESTful APIs are used to connect the frontend and backend components, while authentication and authorization mechanisms ensure that users can access only the resources permitted for their roles.

4.2.1 Guest Access

Visitors can access the public portions of Servanta without registering. They can explore the platform, browse available services and categories, and view relevant service information. Authentication is required when users need to perform actions such as booking services or accessing account-specific functionality.

4.2.2 Administrator Role

The Administrator provides centralized management and oversight of the marketplace. The administrator can monitor platform activity and manage important platform-level information such as users, merchants, services, categories, bookings, and other administrative data.

The administrator role helps maintain the overall organisation and integrity of the marketplace while allowing merchants to independently manage their own operations.

4.2.3 Merchant Role

Merchants can register and operate their own service profiles within the marketplace. They can create and manage service listings, update service information and pricing, manage bookings, and monitor customer interactions through their merchant dashboard.

The multi-tenant structure allows multiple merchants to operate on the same platform without requiring each merchant to maintain a separate application.

4.2.4 Customer Role

Customers can use Servanta to discover available services and merchants through the platform. They can search and filter services, view service details, make bookings, track booking status, complete the available payment workflow, and provide reviews after completing a service.

This provides customers with a centralized interface for managing their service-booking activities.

4.2.5 Service Discovery and Search

Servanta provides service discovery functionality that allows customers to browse available services and categories. Search and filtering capabilities help users find relevant services more efficiently based on the information available on the platform.

This reduces the need for customers to search across multiple independent websites or communication channels.

4.2.6 Booking Management

The booking module provides a structured workflow for interactions between customers and merchants. Customers can submit booking requests, while merchants can view and manage the bookings associated with their services.

Booking information is stored centrally in the PostgreSQL database, allowing users to track the progress and status of their bookings through their respective dashboards.

4.2.7 Payment Workflow

Servanta includes a payment workflow for processing service payments. The implementation uses the **Razorpay test/simulated payment environment**, allowing the payment process to be demonstrated without processing real monetary transactions.

This approach provides a practical demonstration of payment integration while keeping the internship project within a safe testing environment.

4.2.8 Reviews and Feedback

Customers can provide reviews and ratings for completed services. This functionality allows customers to share their experience and provides merchants with useful feedback regarding their services.

Reviews also help improve transparency within the marketplace by giving future customers additional information when evaluating available services.

4.2.9 Authentication and Role-Based Access Control

Servanta uses authentication and authorization mechanisms to protect user accounts and application resources. **JWT-based authentication** is used for maintaining authenticated sessions, while role-based authorization restricts access to features based on whether the user is a Customer, Merchant, or Administrator.

Security measures such as **Helmet, CORS configuration, rate limiting, input validation, and protected API routes** are also incorporated to improve the security and reliability of the application.

4.2.10 Advantages of the Proposed Solution

The proposed Servanta system provides the following advantages:

- Provides independent merchants with a centralized platform to create and manage their service listings.
- Allows customers to discover and compare services from multiple merchants through a single interface.
- Integrates service discovery, booking, payment, and review functionality into one platform.

- Enables merchants to manage their services and bookings through dedicated dashboards.
- Provides administrators with centralized tools for monitoring and managing marketplace activities.
- Reduces dependence on manual booking and communication processes.
- Uses role-based access control to provide appropriate functionality and permissions to different users.
- Provides a scalable multi-tenant structure that can accommodate additional merchants, services, and categories.
- Incorporates security mechanisms to protect users and application resources.
- Provides a structured and user-friendly digital marketplace experience for customers and service providers.

5 Proposed Design / Model

This chapter presents the technical design of **Servanta**, including its system architecture, high-level and low-level design, entity relationships, data flow, application workflow, and database structure. The design was developed to support a multi-tenant service marketplace in which Customers, Merchants, and Administrators interact with the system through role-specific features.

The architecture follows a layered client-server approach. The React frontend handles user interaction, the Node.js and Express.js backend manages application logic and RESTful APIs, and PostgreSQL provides persistent data storage. Authentication, authorization, validation, and security mechanisms are integrated throughout the system to provide controlled and reliable access to application resources.

5.1 System Architecture

Servanta follows a **client-server architecture** consisting primarily of a presentation layer, application layer, and data layer.

The **frontend layer** is developed using React.js with TypeScript and provides the user interface for Customers, Merchants, and Administrators. The frontend communicates with the backend through RESTful HTTP APIs.

The **backend layer** is implemented using Node.js and Express.js. It handles authentication, authorization, business logic, service management, booking operations, payment workflows, reviews, administrative operations, and communication with the database.

The **data layer** uses PostgreSQL to store and manage application data. The backend communicates with PostgreSQL to perform database operations and maintain relationships between users, merchants, services, bookings, payments, reviews, and other entities.

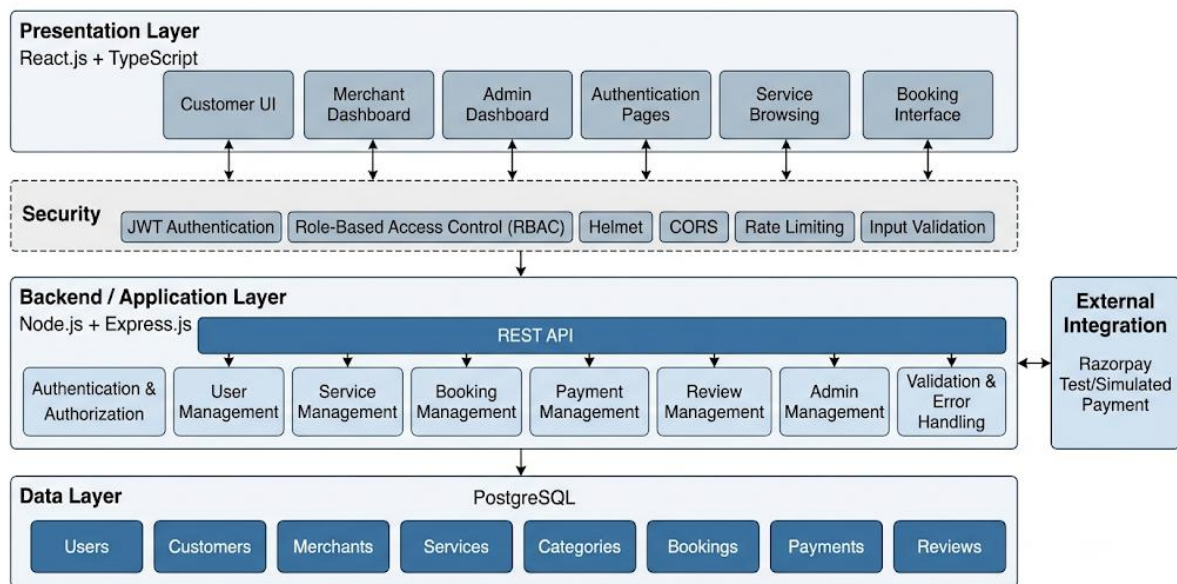


Figure 5.1: System Architecture Overview

5.2 High Level Diagram

The high-level design represents Servanta as a set of cooperating layers and functional modules.

The **presentation layer** provides the user interface through React components and role-specific dashboards. Customers, Merchants, and Administrators interact with the system through these interfaces.

The **application layer** contains the Express.js REST API and the main business modules, including:

- Authentication and user management
- Merchant management
- Customer management
- Service and category management
- Service search and filtering
- Booking management
- Payment processing
- Review and rating management
- Administrative management
- Security and authorization

The **data layer** consists of the PostgreSQL database, which stores the persistent information required by these modules.

Role-based access control determines which functionality each type of user can access. Customers can access customer-specific operations, Merchants can manage their own services and bookings, and Administrators have platform-level management capabilities.

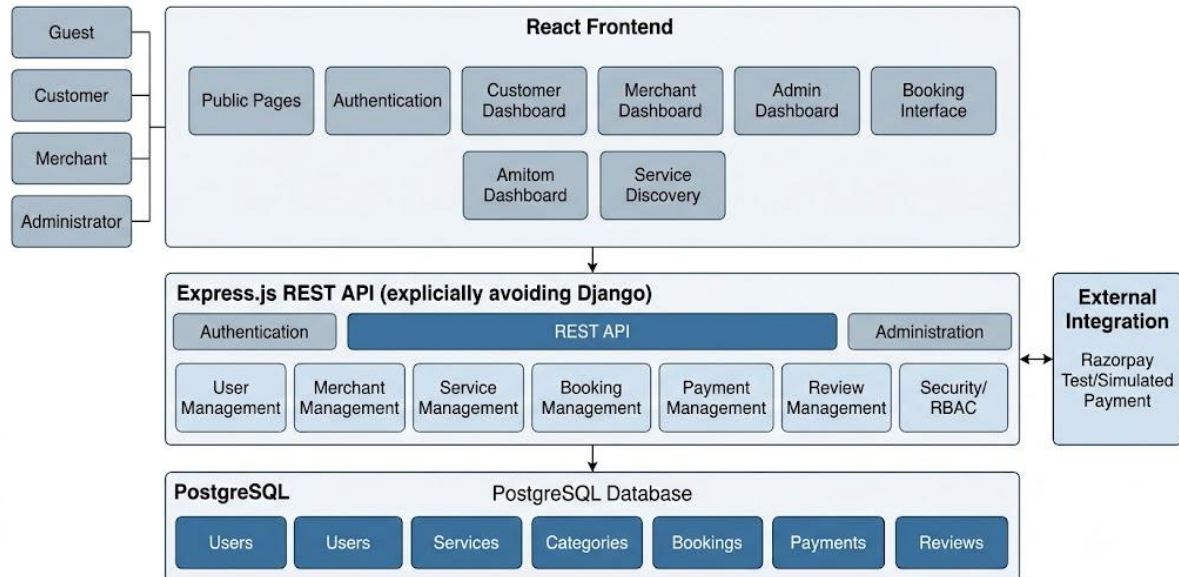


Figure 5.2: High-Level System Diagram

5.3 Low Level Diagram

The low-level design describes the individual modules and their interactions within the Servanta application.

The React frontend is organised into components, pages, forms, dashboards, and role-specific interfaces. These frontend components communicate with corresponding Express.js API endpoints.

The backend is divided into functional areas such as authentication, users, merchants, services, categories, bookings, payments, reviews, and administrative operations. Each module contains the necessary API routes, controllers, middleware, validation, and business logic required to perform its operations.

For example, when a Customer submits a booking request, the React frontend sends the booking information to the appropriate backend API. The Express.js backend authenticates the user, validates the request, verifies the required information, processes the booking logic, and stores the booking in PostgreSQL.

This modular structure separates different functional responsibilities and makes the system easier to maintain, test, debug, and extend.

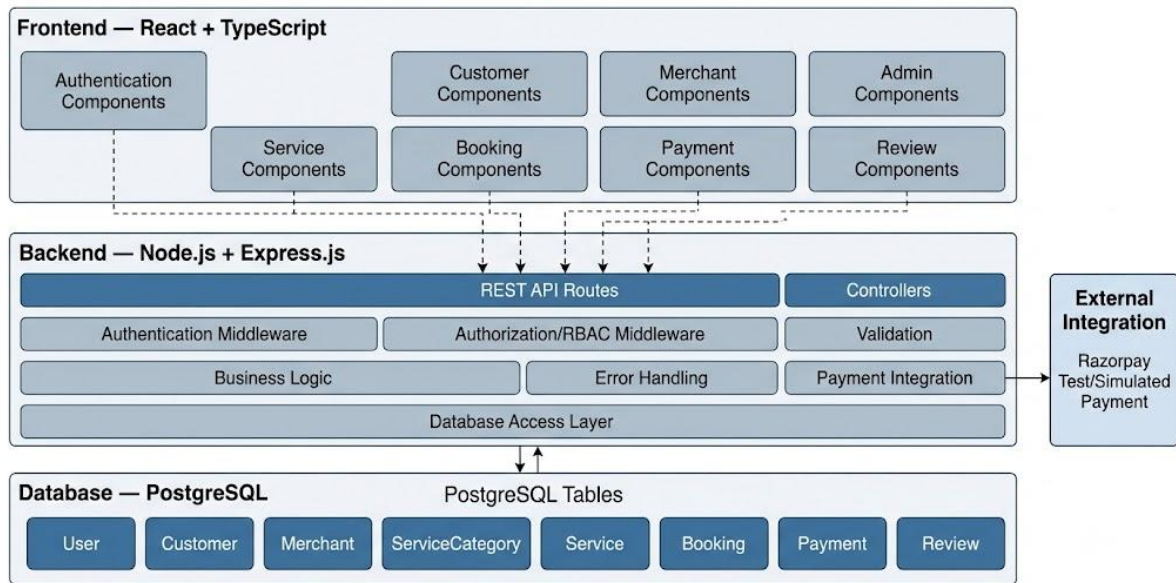


Figure 5.3: Low-Level Module Diagram

5.4 Entity-Relationship Diagram

The Entity-Relationship Diagram represents the major entities and relationships used within the Servanta database.

The primary entities include **User, Merchant, Customer, Service Category, Service, Booking, Payment, and Review**, along with other supporting entities required by the application.

A User represents an authenticated account within the platform and is associated with a specific role. Depending on the role, the account can be associated with Customer, Merchant, or Administrator functionality.

A Merchant can manage multiple Services, while each Service belongs to an appropriate category and is associated with its respective Merchant. Customers can create multiple Bookings, while each Booking connects a Customer with a selected Service and its Merchant.

Payment information is associated with the corresponding booking, allowing the system to maintain the payment status of a service transaction. Reviews are associated with completed service interactions, allowing customers to provide ratings and feedback.

These relationships ensure that the marketplace data remains structured and that transactions can be traced between users, services, bookings, payments, and reviews.

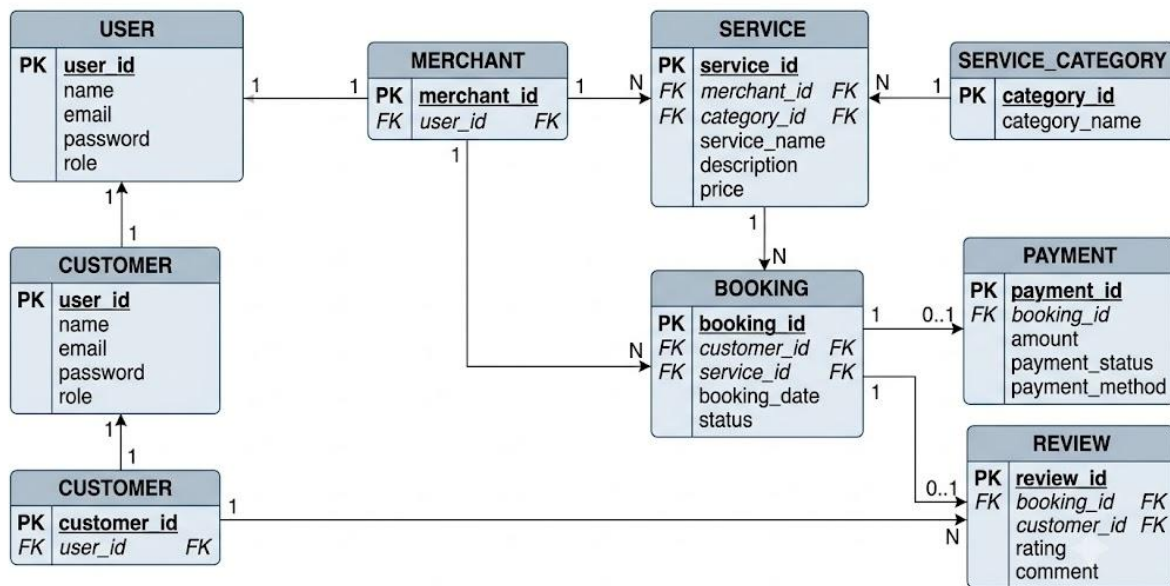


Figure 5.4: Entity-Relationship Diagram

5.5 Data Flow Diagram

The Data Flow Diagram illustrates how information moves through Servanta during important operations.

For the central booking workflow, a Customer first accesses the React frontend and searches or browses available services. After selecting a service, the Customer submits a booking request through the frontend.

The request is sent to the Express.js REST API, where authentication and authorization checks are performed. The backend validates the request and communicates with PostgreSQL to create and store the booking information.

The Merchant can then access the booking through the merchant dashboard and process the request. Depending on the booking workflow, the Customer can proceed with the available payment process. The payment information is processed through the payment workflow and the corresponding booking and payment records are updated.

After the service is completed, the Customer can submit a review and rating. Relevant information is stored in the database and can be accessed through the appropriate user interfaces.

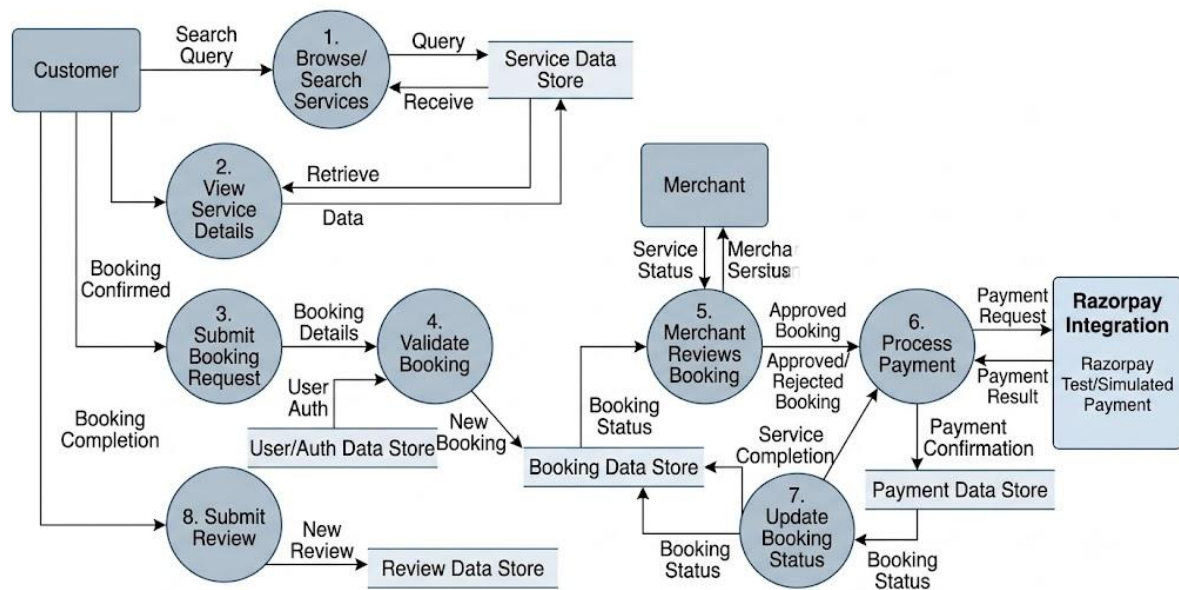


Figure 5.5: Data Flow Diagram – Booking and Payment Flow

5.6 Flowchart

The Servanta booking workflow begins when a Customer searches for and selects a service. The Customer provides the required booking information and submits the booking request.

The system validates the request and creates the booking. The Merchant can view the request through the merchant dashboard and take the appropriate action.

If the booking is accepted, the Customer can proceed with the payment workflow. After successful completion of the test/simulated payment process, the booking is updated accordingly.

Once the service has been delivered and the booking reaches the completed stage, the Customer can provide a rating and review for the service.

The simplified workflow can be represented as:

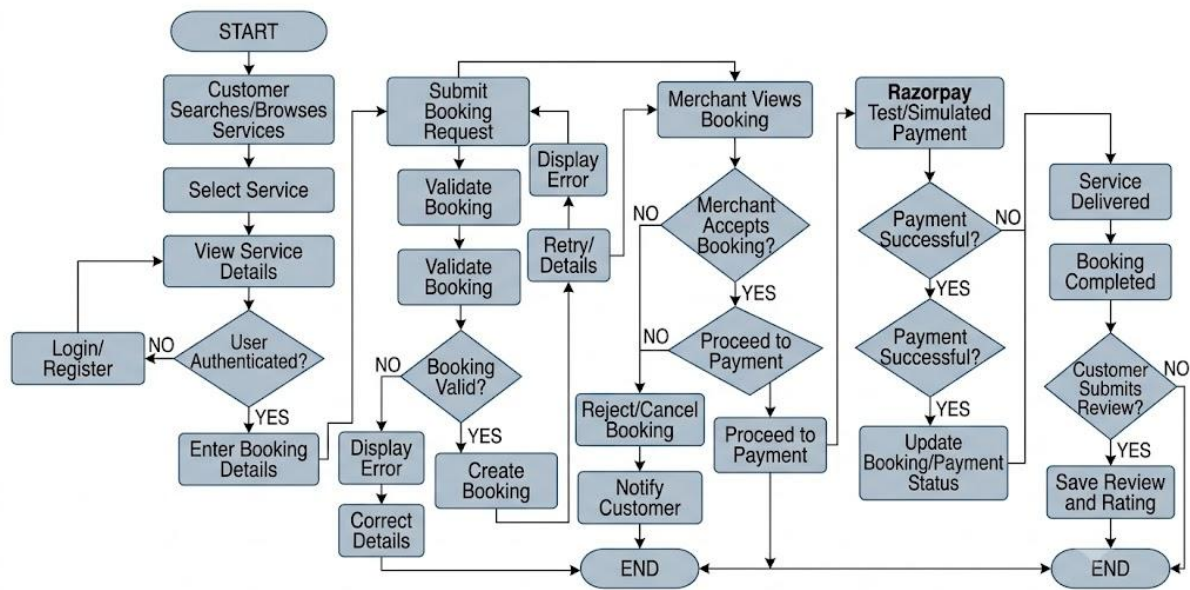


Figure 5.6: Booking Process Flowchart

5.7 Database Design

The Servanta database is designed using **PostgreSQL** and consists of related entities that support the marketplace's users, services, transactions, and administrative operations.

The **User** entity stores common account information and authentication-related data. Role information determines whether the account operates as a Customer, Merchant, or Administrator.

The **Merchant** entity stores merchant-specific information and is associated with the services managed by the merchant. A Merchant can create and manage multiple service listings.

The **Customer** entity stores customer-specific information and is associated with the bookings created by the customer.

The **Service Category** entity is used to organise services into logical categories. A category can contain multiple services.

The **Service** entity contains information about individual services offered by merchants, including relevant details such as service name, description, pricing, category, and merchant association.

The **Booking** entity acts as a central transactional entity. It connects a Customer with a selected Service and stores information related to the booking, including its status and relevant booking details.

The **Payment** entity stores payment-related information associated with bookings. Servanta uses a Razorpay test/simulated payment workflow, allowing the payment process to be demonstrated without processing real monetary transactions.

The **Review** entity stores customer feedback and ratings associated with completed service interactions. This allows feedback to remain connected to the relevant service and booking.

Foreign key relationships are used to maintain connections between related entities and preserve **referential integrity**. For example, a booking is associated with a valid customer and service, while a payment is associated with its corresponding booking.

The database structure is organised to minimise unnecessary duplication and maintain consistency between related records. Separating entities such as users, services, categories, bookings, payments, and reviews into dedicated tables allows the application to manage marketplace information efficiently.

The PostgreSQL database is accessed by the Node.js and Express.js backend through the application's database layer. REST API operations perform the required Create, Read, Update, and Delete operations while application-level validation and database constraints help maintain data integrity.

This database design provides a structured foundation for the multi-tenant nature of Servanta and allows the system to support multiple merchants, customers, services, and transactions within a single centralized marketplace.

6 PROJECT MODULES

Servanta was developed as a collection of interconnected functional modules, with each module responsible for a specific area of the service marketplace. The modules work together through the React frontend, Node.js and Express.js backend, RESTful APIs, and PostgreSQL database. This chapter describes the purpose, implementation, workflow, user interface, and possible future enhancements of each major module.

6.1 Authentication Module

Purpose:

Provides a secure entry point for Customers, Merchants, and Administrators and ensures that authenticated users can access the functionality appropriate to their roles.

Implementation:

The authentication system is implemented through the Node.js and Express.js backend using **JWT-based authentication**. User credentials and role information are managed through the PostgreSQL database. Passwords are securely handled using password hashing, while protected API routes use authentication middleware to verify valid tokens. Role-based authorization is used to restrict access to role-specific functionality.

Workflow:

A new user registers by providing the required information and selecting the appropriate account type. During login, the backend validates the credentials and generates an authentication token. The frontend uses the authenticated session to provide access to the relevant Customer, Merchant, or Administrator interface.

Screens:

Login page, registration page, authentication validation messages, and role-specific access interfaces.

Future Enhancements:

Email verification, password-reset functionality, refresh-token authentication, and optional two-factor authentication.

6.2 Administrator Module

Purpose:

Provides centralized management and monitoring capabilities for maintaining the overall Servanta marketplace.

Implementation:

The Administrator module is implemented using protected Express.js API endpoints

and a dedicated React-based administrator interface. Role-based authorization ensures that administrative operations can only be accessed by authorized users.

Workflow:

The administrator logs into the platform and accesses the administrative dashboard. From there, the administrator can manage users, merchants, service categories, services, bookings, and relevant platform activities according to the implemented permissions.

Screens:

Administrator Dashboard, user management interface, merchant/service management interface, category management interface, and platform activity views.

Future Enhancements:

Advanced analytics, dispute management, automated moderation, detailed revenue reports, and platform-level audit reporting.

6.3 Merchant Module

Purpose:

Allows independent merchants to establish and manage their presence on Servanta while keeping their operational data separated from other merchants.

Implementation:

Merchant-specific information is stored in PostgreSQL and associated with the authenticated merchant account. Express.js APIs handle merchant operations, while React components provide the merchant dashboard and management interfaces. Authorization ensures that merchants can manage only the resources associated with their accounts.

Workflow:

A merchant registers and accesses the merchant dashboard after authentication. The merchant can create and manage service listings, update service information, view incoming bookings, and manage booking requests associated with their services.

Screens:

Merchant Dashboard, service management interface, service creation/editing form, and booking management view.

Future Enhancements:

Merchant analytics, promotional pricing, bulk service management, availability calendars, and advanced revenue reporting.

6.4 Customer Module

Purpose:

Provides customers with a centralized interface for discovering services, making bookings, managing transactions, and interacting with merchants.

Implementation:

Customer information and activity are stored in PostgreSQL. React components provide customer-facing interfaces, while Express.js APIs handle customer operations such as service browsing, booking management, payments, and reviews.

Workflow:

A customer registers or logs into the platform, browses available services, searches and filters listings, views service details, submits bookings, tracks booking status, completes the available payment workflow, and provides reviews for completed services.

Screens:

Customer Dashboard, service browsing page, search/filter interface, service details page, booking management page, payment interface, and review interface.

Future Enhancements:

Favourite services, personalized recommendations, loyalty programmes, saved searches, and improved customer profiles.

6.5 Service Management Module

Purpose:

Allows merchants to create and manage the services they offer while maintaining a structured service catalogue through platform-level categories.

Implementation:

Services and service categories are stored in PostgreSQL. Each service is associated with its respective merchant and category. Express.js REST APIs perform service creation, retrieval, updating, and deletion operations, while React provides the corresponding management and customer-facing interfaces.

Workflow:

A merchant creates a service by entering relevant information such as service name, description, category, and price. The information is validated and stored in PostgreSQL. The service then becomes available through the customer-facing service catalogue. Merchants can subsequently update or remove their own services.

Screens:

Add Service form, Edit Service form, Merchant Service Management page, Service Catalogue, and Service Details page.

Future Enhancements:

Service variants and add-ons, advanced availability management, service image galleries, and merchant-defined scheduling options.

6.6 Booking Module

Purpose:

Manages the central transaction between customers and merchants by allowing customers to request services and merchants to manage those requests.

Implementation:

Booking information is stored in PostgreSQL and associated with the relevant Customer and Service. Express.js APIs handle booking creation and status updates, while authorization middleware ensures that customers and merchants can access only the bookings relevant to them.

Workflow:

A customer selects a service and submits a booking request with the required details. The booking is stored in the database and becomes available to the relevant merchant. The merchant can review and update the booking status. Customers can track the progress of their bookings through their dashboard.

Screens:

Booking form, Customer Booking page, Merchant Booking Management page, and Booking Status view.

Future Enhancements:

Advanced calendar-based availability, automatic conflict detection, booking reminders, rescheduling, and automated notifications.

6.7 Payment Module

Purpose:

Provides a payment workflow for service bookings while allowing the project to demonstrate payment integration without processing real monetary transactions.

Implementation:

Servanta integrates a **Razorpay test/simulated payment workflow**. The backend

manages payment-related operations and associates payment information with the relevant booking. Payment status and transaction-related information can be maintained in PostgreSQL.

Workflow:

After a booking reaches the appropriate stage, the customer proceeds to the payment interface. The payment workflow is executed using the Razorpay test environment. After the test payment process returns a result, the relevant payment and booking information is updated accordingly.

Screens:

Payment initiation/checkout interface, payment result/status page, and payment-related booking information.

Future Enhancements:

Production payment gateway integration, refunds, partial payments, multiple payment methods, invoices, and automated payment reconciliation.

6.8 Review Module

Purpose:

Allows customers to provide ratings and feedback regarding services they have completed, helping improve transparency and service quality within the marketplace.

Implementation:

Review information is stored in PostgreSQL and associated with the relevant booking and customer. Express.js APIs handle review creation and retrieval, while React components display reviews on relevant service interfaces.

Workflow:

After a booking has been completed, the customer can submit a rating and written feedback. The review is validated and stored in the database. Relevant reviews can then be displayed to users viewing the associated service.

Screens:

Review submission interface, service reviews section, and customer review history.

Future Enhancements:

Merchant replies, review moderation, verified-booking indicators, review reporting, and review analytics.

6.9 Notification Module

Purpose:

Provides users with relevant information regarding important marketplace activities such as booking status changes, payment events, and other account-related events.

Implementation:

Notification-related functionality is integrated with the backend workflow so that relevant events can be communicated to the appropriate users. The React frontend can display these notifications within the authenticated user interface.

Workflow:

When an important event occurs, such as a change in booking status or a payment-related update, the backend generates or provides the relevant notification information. The appropriate user can then view the information through their application interface.

Screens:

Notification area/list within the Customer and Merchant interfaces.

Future Enhancements:

Real-time notifications using WebSockets, email notifications, SMS alerts, push notifications, and notification preferences.

6.10 Dashboard Module

Purpose:

Provides a centralized, role-specific interface from which users can access the functions available to their account type.

Implementation:

Dashboards are developed using React.js and TypeScript. Separate interfaces are provided for Customers, Merchants, and Administrators. The information displayed on each dashboard is retrieved through protected Express.js REST APIs according to the authenticated user's role and permissions.

Workflow:

After successful authentication, the user is directed to the appropriate dashboard. The Customer dashboard provides access to services, bookings, payments, and reviews. The Merchant dashboard provides access to services and booking management. The Administrator dashboard provides centralized management and monitoring functionality.

Screens:

Customer Dashboard, Merchant Dashboard, and Administrator Dashboard.

Future Enhancements:

Interactive analytics, customizable dashboard widgets, real-time activity feeds, advanced charts, and personalized dashboard layouts.

6.11 Security Module

Purpose:

Protects the Servanta application, user accounts, APIs, and sensitive operations against common web application security risks.

Implementation:

Security is implemented across the frontend and backend using multiple mechanisms. JWT-based authentication protects authenticated resources, while role-based access control restricts sensitive operations. **Helmet** is used to add appropriate HTTP security headers, **CORS** is configured to control cross-origin requests, and **express-rate-limit** helps protect API endpoints against excessive requests. Input validation and protected routes are also used to improve application security.

Workflow:

When a request reaches a protected API endpoint, the backend verifies authentication and authorization before allowing the requested operation. Input data is validated before processing, while security middleware provides additional protection at the HTTP/API level.

Screens:

The security module primarily operates at the backend/API level, while users may encounter authentication, authorization, validation, or error messages when an operation is not permitted.

Future Enhancements:

Two-factor authentication, stronger password policies, security logging, automated vulnerability scanning, refresh-token rotation, and advanced monitoring.

7 IMPLEMENTATION

This chapter describes how the Servanta project was organised and configured as a working full-stack application, covering the implementation aspects that support the individual functional modules described in the previous chapter.

7.1 Folder Structure and Project Organisation

The project follows a structured full-stack organisation, with the React.js frontend and Node.js/Express.js backend maintained as separate application layers. The frontend contains reusable components, pages, dashboards, and role-specific interfaces, while the backend contains routes, controllers, middleware, validation, and database-related logic. This separation kept development organised as the different marketplace modules were implemented and integrated.

7.2 React Components and Frontend Implementation

The frontend is developed using React.js with TypeScript. Reusable components are used for common interface elements, while separate pages and dashboard components provide functionality for Customers, Merchants, and Administrators. React routing is used to manage navigation between pages, and API requests connect the frontend interfaces with the Express.js backend.

7.3 Environment Variables and Configuration

Sensitive configuration values, including database credentials, authentication-related secrets, and payment configuration, are maintained using environment variables rather than being directly stored in the source code. Separate configuration values are used for the development environment, helping keep sensitive information outside the main application codebase.

7.4 Version Control with Git and GitHub

Git was used for version control throughout the development of Servanta, with the project maintained in a GitHub repository. A .gitignore file was used to exclude dependencies, environment files, and other files that should not be committed to the repository. Changes were committed incrementally during development, providing a history of the project's implementation and updates.

7.5 PostgreSQL Database and Backend Configuration

PostgreSQL was configured as the project's relational database and stores the application's user, merchant, customer, service, booking, payment, and review information. The Node.js backend communicates with PostgreSQL through the database layer, while database connection details are supplied through environment variables.

7.6 Project Organisation and Code Reusability

The Servanta codebase was organised around reusable React components, backend middleware, API routes, and shared application logic. Common frontend elements were implemented as reusable components rather than duplicated across different pages. Similarly, authentication, authorization, validation, and security middleware are reused across protected API routes, helping keep the codebase maintainable and consistent as additional marketplace functionality was integrated.

8 TESTING AND PERFORMANCE EVALUATION

Testing was carried out throughout the development of Servanta, with more extensive testing performed after the major modules had been integrated into a single application. The testing process focused on verifying the functionality, security, API communication, database operations, user interface behaviour, and overall reliability of the marketplace.

8.1 Test Plan / Test Cases

Test cases were grouped according to the major functional areas of Servanta, covering authentication and authorization, service management, booking operations, payment workflow, reviews, API functionality, database integrity, role-based access control, user interface behaviour, input validation, and security mechanisms. Representative test cases from each functional area are summarised in the tables below.

8.1.1 Authentication Testing

Authentication testing was performed to verify the registration, login, authentication, and role-based authorization functionality of Servanta. New Customer and Merchant accounts were successfully registered with the correct roles, and valid credentials redirected users to their respective dashboards. Invalid login credentials were rejected with appropriate error messages. Attempts to access Administrator functionality using unauthorized roles were also blocked successfully, confirming that role-based access control was functioning as expected.

8.1.2 CRUD Testing

CRUD testing was performed on the major data-management operations of the application. Merchants were able to create, edit, and delete their service listings successfully, with changes reflected in the customer-facing service catalogue. Administrator operations such as adding service categories were also tested and verified. The tests confirmed that the corresponding create, read, update, and delete operations were correctly processed through the backend APIs and stored in PostgreSQL.

8.1.3 Booking Testing

The booking workflow was tested from both Customer and Merchant perspectives. Customers were able to submit booking requests successfully, after which the bookings were stored with the appropriate status. Merchants were able to review and update booking requests, including approving, rejecting, and completing bookings. The corresponding status changes were correctly reflected in the customer and merchant interfaces, confirming that the booking workflow operated as intended.

8.1.4 Payment Testing

Payment testing focused on the Razorpay test/simulated payment workflow integrated into Servanta. Customers were able to initiate the payment process for eligible bookings, and successful test transactions were recorded against the corresponding booking. Payment-related information was also verified through the relevant application interfaces. Since the project uses the Razorpay test environment, the testing was performed without processing real monetary transactions.

8.1.5 Review Testing

The review functionality was tested to ensure that customers could provide feedback for completed service bookings. Reviews were successfully submitted and displayed for the relevant services. The system also prevented inappropriate review actions where the required booking conditions were not satisfied. These tests confirmed that review information was correctly validated, stored, and retrieved through the backend APIs.

8.1.6 UI and Cross-Browser Testing

User interface testing was performed to verify the usability and consistency of the React-based application across different screen sizes and browsers. The major dashboards, service pages, booking interfaces, and forms were checked for correct rendering and navigation. The application was tested on desktop and mobile-sized screens, and major pages were checked using commonly used browsers such as Google Chrome and Microsoft Edge. The interfaces rendered correctly during testing, with responsive layouts adapting to different screen sizes.

8.1.7 Database and Validation Testing

Database and validation testing was performed to verify data integrity and ensure that invalid or incomplete information was not incorrectly stored. PostgreSQL relationships and constraints were tested during operations involving users, services, bookings, payments, and reviews. Registration and other forms were tested with missing, invalid, and duplicate information, and appropriate validation responses were displayed. These tests confirmed that application-level validation and database constraints worked together to maintain consistent and reliable data.

8.2 Test Procedure

Each test case was executed manually against the running Servanta application, first to verify individual features and then again after the major modules had been integrated to ensure that interactions between modules did not introduce unexpected issues. Authentication and role-based access control were repeatedly tested whenever protected API routes or role-specific functionality was added, since incorrect authorization could potentially allow users to access resources outside their permitted role.

8.3 Performance Evaluation

The application remained responsive throughout testing, with the main dashboards, service listings, and booking pages loading without noticeable delays under the data volumes used during development. Backend API operations and PostgreSQL queries were reviewed during the final stages of development to ensure that commonly used operations such as retrieving services, bookings, payments, and reviews were handled efficiently.

Authentication and JWT-based authorization performed consistently across repeated tests, with protected routes correctly accepting valid requests and rejecting unauthorized access. Navigation between the Customer, Merchant, and Administrator interfaces was verified to function correctly, and the React-based interface was tested across different screen sizes and commonly used browsers. Security mechanisms including rate limiting, CORS configuration, Helmet, and input validation were also verified during testing. Overall, the results indicate that Servanta provides a stable and functional platform suitable for demonstration and further development beyond the scope of the internship.

9 GITHUB REPOSITORY

Version control was maintained using Git and GitHub throughout the development of Servanta, providing a centralized repository for the source code and a clear record of the project's development and implementation progress.

9.1 Repository Structure

The repository is organised according to the full-stack architecture of Servanta, with separate frontend and backend sections. The frontend contains React components, pages, dashboards, routing, and user interface functionality, while the backend contains Express.js routes, controllers, middleware, validation, authentication, and database-related logic. Configuration files, environment templates, documentation, and project setup files are also maintained within the repository.

9.2 Version Control and Commit History

Git was used to track changes throughout the development of the project. Commits were made incrementally as major features and improvements were implemented, including authentication, merchant and customer functionality, service management, booking, payment integration, reviews, security improvements, search and filtering, and production-readiness features. The commit history provides a record of the project's progression from initial setup through to the completed application.

9.3 README

The README provides an overview of Servanta, its purpose, technology stack, project structure, and instructions for setting up and running the application. It includes the required environment configuration, database setup information, frontend and backend installation steps, and other relevant instructions needed to understand and run the project locally.

9.4 Repository Link

<https://github.com/angel-mariya-tj/upskillcampus.git>

10 SCREENSHOTS

The following pages present representative screenshots of the completed **Servanta** application, captured from the running system and covering the major interfaces and core functionalities across the **Customer, Merchant, and Administrator** roles. The screenshots demonstrate the application's user interface, service management, booking workflow, payment process, and other key features implemented during the internship.

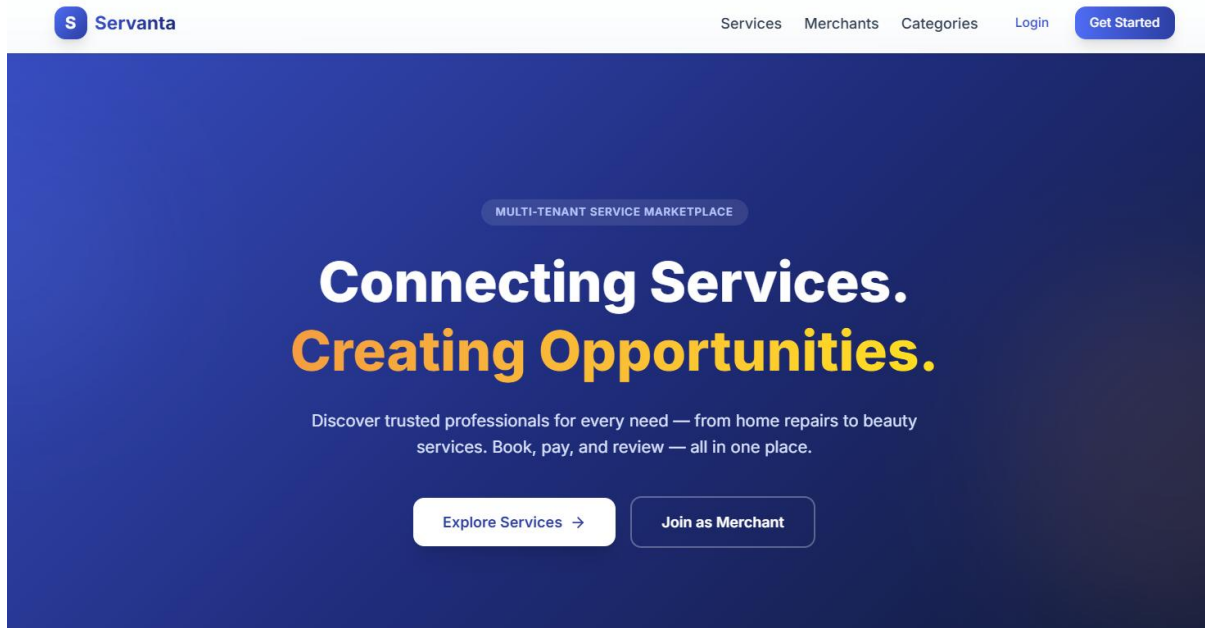


Figure 10.1: Home Page



Welcome back
Sign in to your Servanta account

Email

you@example.com

Password

Sign In

Don't have an account? [Create one](#)

Figure 10.2: Login

S Servanta

Services Merchants Categories Login

Get Started



Create an account
Join Servanta today

Full Name

John Doe

Email

aravind1@servanta.com

Phone

+91 98765 43210

Password

I want to

☐ Book Services
 ☐ Offer Services

Create Account

Already have an account? [Sign in](#)

Figure 10.3: Registration

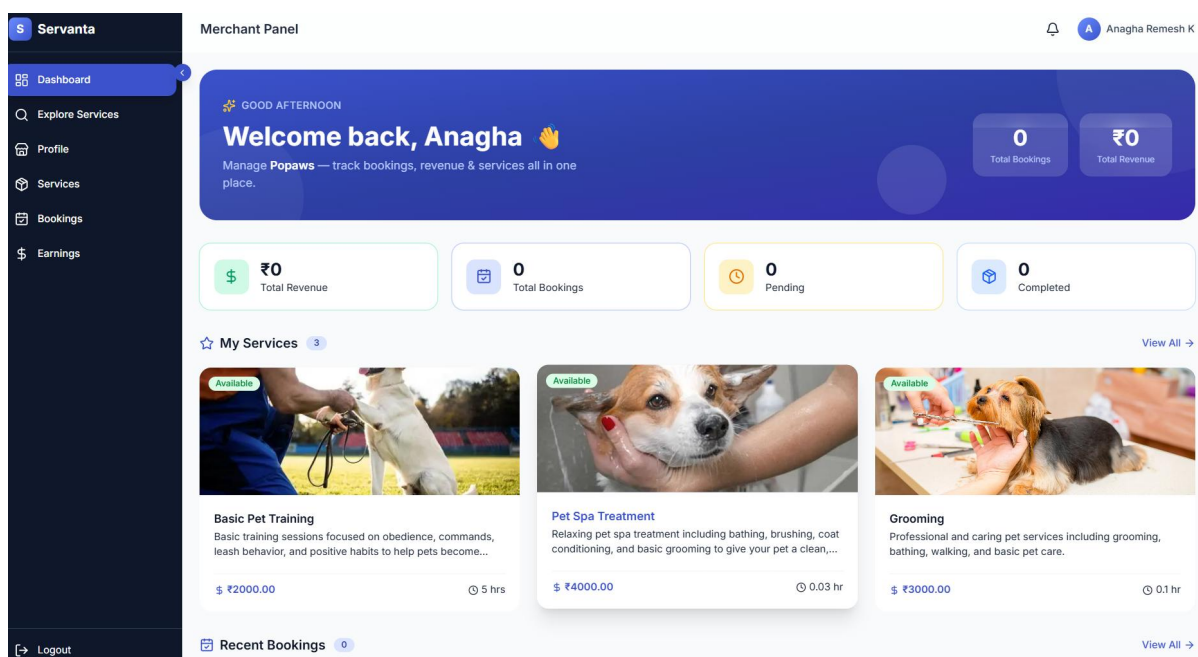


Figure 10.4: Merchant Dashboard

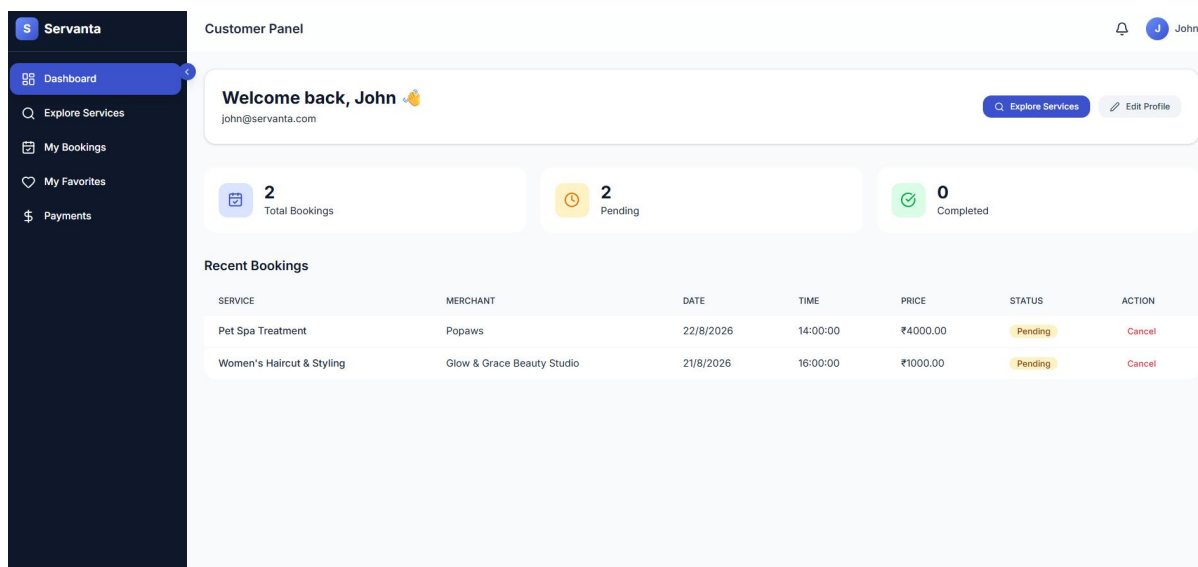


Figure 10.5: Customer Dashboard

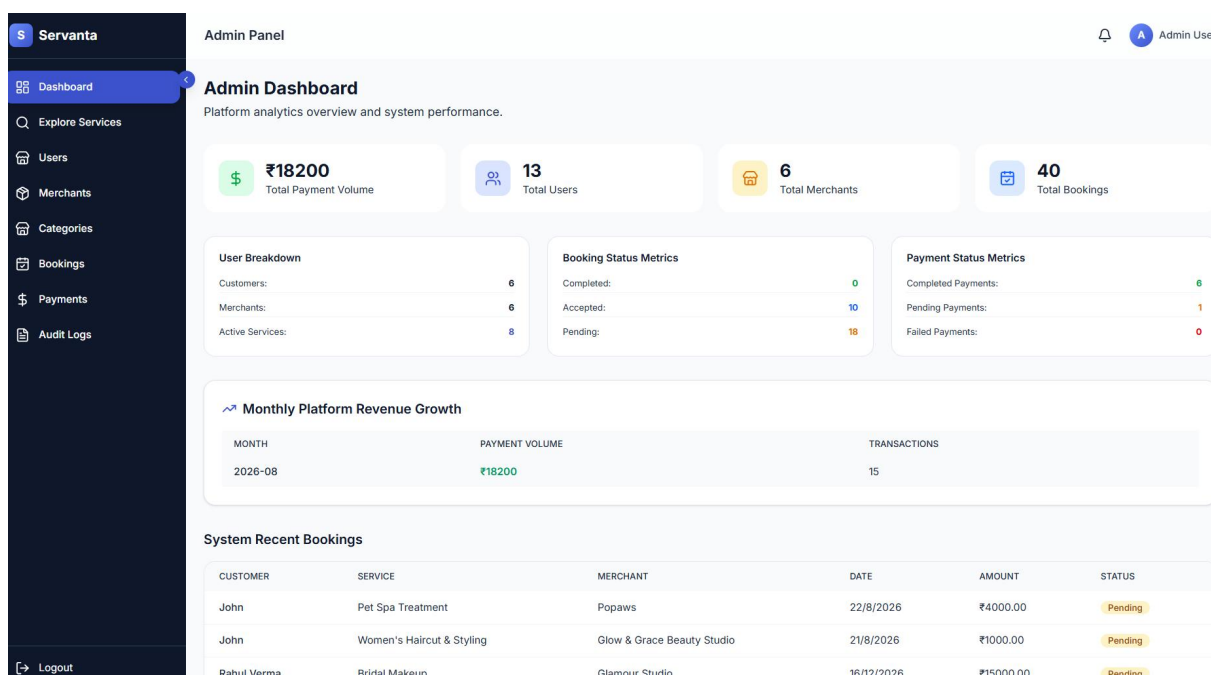


Figure 10.6: Admin Dashboard

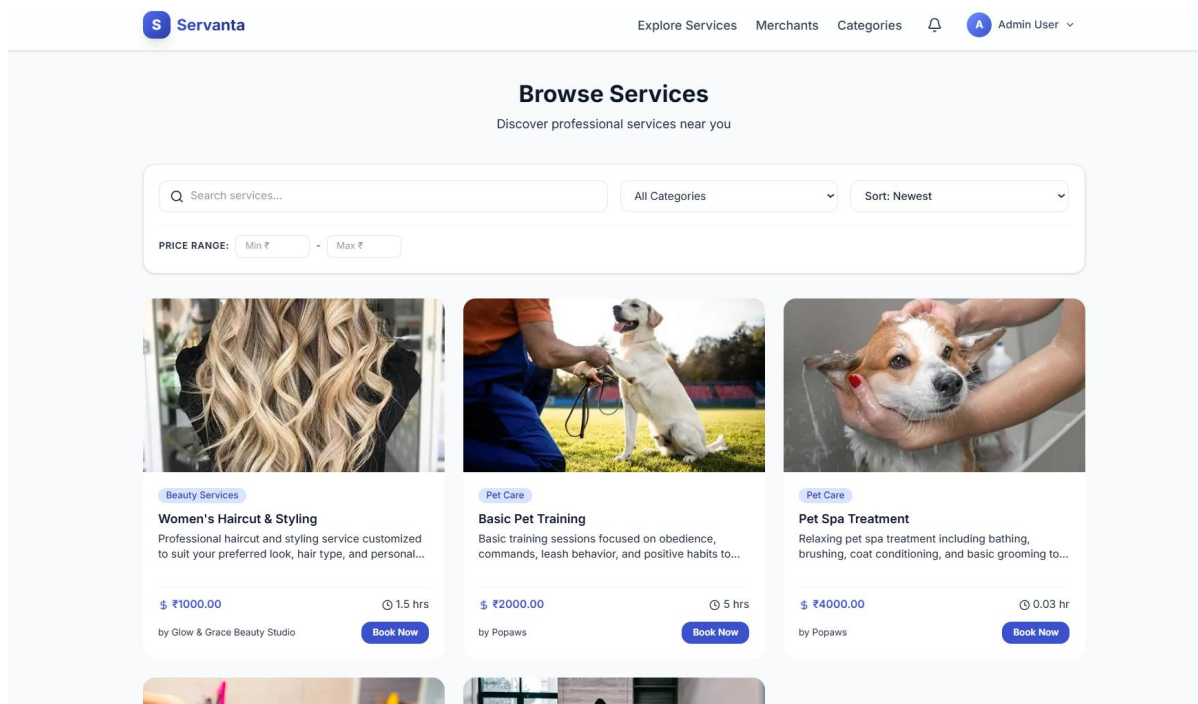


Figure 10.7: Service Listing

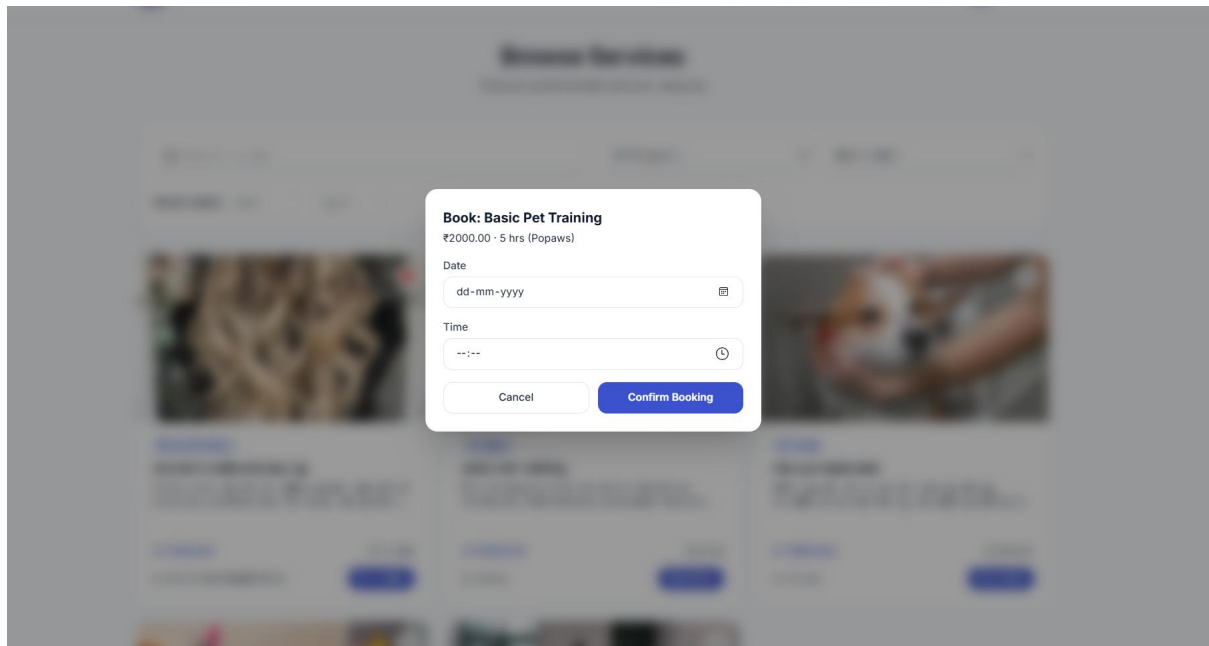


Figure 10.8: Booking

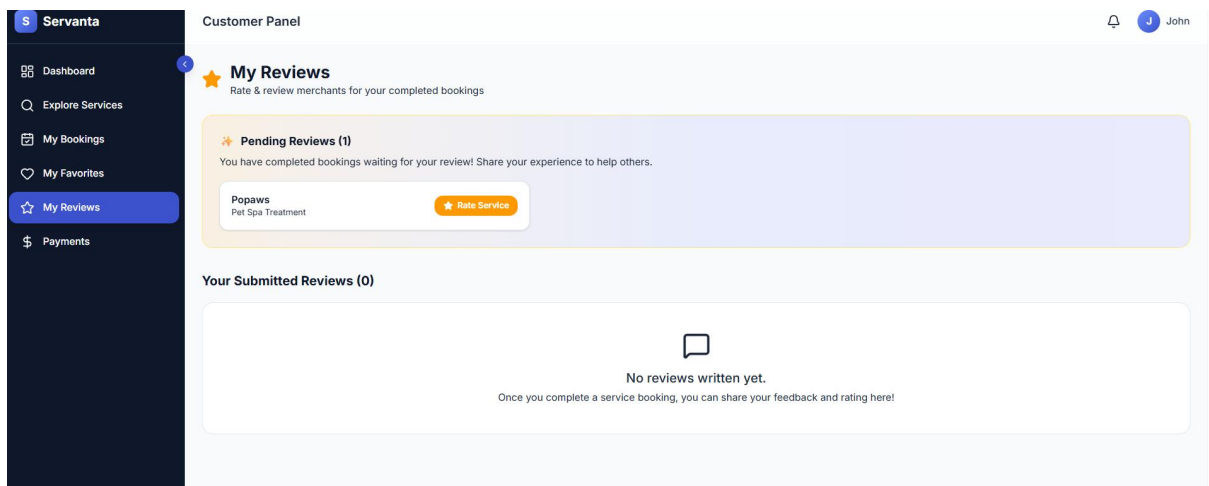


Figure 10.10: Reviews

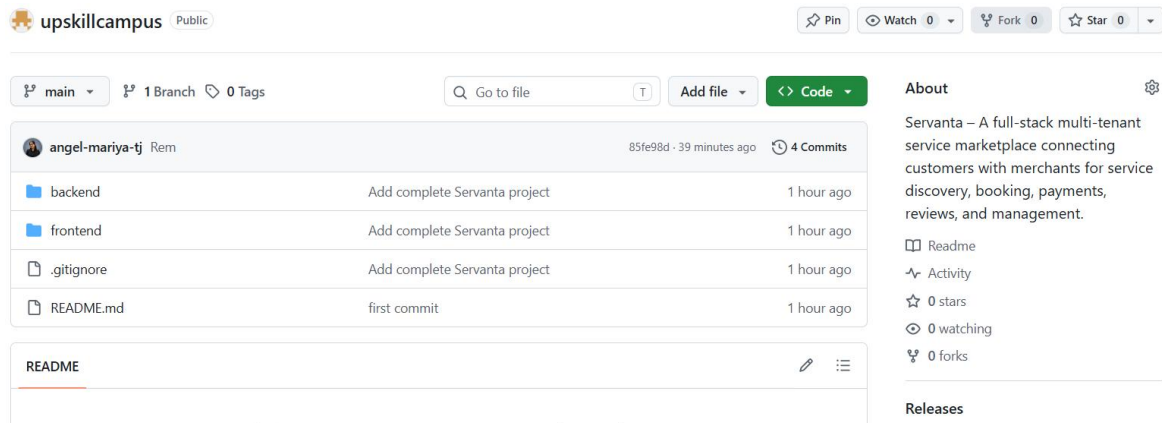


Figure 10.12: GitHub Repository

11 MY LEARNINGS

The internship's greatest value came from bringing together multiple technologies and software engineering concepts into a single, working full-stack system. Developing Servanta provided practical experience in frontend development, backend API development, database management, authentication, security, testing, and project organisation. The following sections summarise the major technical and professional learnings gained during the internship.

React.js and TypeScript: Developing the Servanta frontend helped me understand component-based architecture and the importance of reusable UI components. Using TypeScript also provided a better understanding of type safety and helped reduce errors while working with application data and API responses.

Node.js and Express.js: Building the backend provided practical experience in creating RESTful APIs, handling HTTP requests, implementing business logic, and organising backend functionality into routes, controllers, middleware, and supporting services.

PostgreSQL and Database Design: Designing the relational database helped me understand how users, merchants, services, bookings, payments, and reviews should be connected through appropriate relationships. Working with PostgreSQL also highlighted the importance of data integrity, constraints, and efficient database operations.

Authentication and Access Control: Implementing JWT-based authentication and role-based access control taught me how to protect application resources and ensure that Customers, Merchants, and Administrators can access only the functionality intended for their roles.

CRUD Operations: Implementing create, read, update, and delete operations across services, categories, users, and other resources made the common patterns of full-stack application development much clearer and demonstrated how frontend interfaces, backend APIs, and database operations work together.

API Integration: Connecting the React frontend with the Express.js backend through REST APIs gave me practical experience in client-server communication, request handling, JSON data exchange, error handling, and managing asynchronous operations.

Payment Integration: Working with the Razorpay test/simulated payment workflow provided an understanding of how third-party payment services interact with a web application and how payment information can be connected with booking transactions without processing real payments during development.

Security: Implementing security mechanisms such as Helmet, CORS configuration, rate limiting, input validation, JWT authentication, and protected routes helped me

understand that security needs to be considered throughout application development rather than added only after the main functionality is complete.

Git and GitHub: Using Git and GitHub throughout development taught me the importance of incremental commits, version tracking, repository organisation, and maintaining a clear history of changes while working on a software project.

Testing and Debugging: Testing individual modules and then testing the complete integrated application showed me that a feature can work correctly in isolation while still producing problems when combined with other modules. This reinforced the importance of integration testing and systematic debugging.

Software Engineering and Project Organisation: Structuring Servanta into separate frontend components, backend modules, API routes, middleware, and database entities helped me understand how proper organisation improves maintainability and makes it easier to introduce new functionality without unnecessarily affecting existing features.

Problem Solving: Developing a complete marketplace from an initial problem statement required decisions about architecture, database relationships, authentication, user roles, workflows, and technology choices. This improved my ability to analyse requirements and convert them into practical technical solutions.

Professional Development and Industrial Practices: Working within a defined internship schedule and maintaining project documentation provided experience in planning, incremental development, testing, version control, and technical reporting. The internship helped me understand how academic concepts are applied when developing a complete software product.

12 FUTURE WORK SCOPE

Servanta was developed within the defined internship scope, while its modular architecture provides opportunities for further development. The following enhancements could be considered if the project is continued beyond the internship:

- Deploying the complete application to a cloud platform such as AWS, Azure, or another suitable hosting provider for public accessibility.
- Integrating production-ready payment processing with a live payment gateway after completing the required security and business requirements.
- Expanding the refund and cancellation system with more flexible merchant and customer policies.
- Adding real-time notifications using WebSockets for booking, payment, and other important events.
- Introducing email and SMS notifications for booking confirmations, reminders, cancellations, and payment updates.
- Adding advanced analytics dashboards for merchants and administrators covering bookings, revenue, services, and customer activity.
- Developing a recommendation system to suggest relevant services to customers based on their preferences and previous activity.
- Adding location-based service discovery and map integration to help customers find nearby merchants.
- Developing a mobile application for Customers and Merchants using the existing backend APIs.
- Adding advanced merchant availability and calendar management to reduce scheduling conflicts.
- Improving search functionality with more advanced ranking, filtering, and personalised results.
- Introducing additional monitoring, logging, automated testing, and deployment automation for production-scale operation.

13 CONCLUSION

This internship took **Servanta** from an initial service marketplace problem statement to a working full-stack web application over the course of the internship. The project covered requirement analysis, system architecture, database design, authentication, role-based access control, service management, booking, payment integration, reviews, security improvements, testing, and documentation.

Technically, the project provided practical experience across the complete application stack: **React.js and TypeScript** for the frontend, **Node.js and Express.js** for the backend, and **PostgreSQL** for persistent data storage. RESTful APIs connected the frontend and backend, while JWT authentication and role-based authorization controlled access to the system. Additional features such as Razorpay test payment integration, search and filtering, security middleware, and production-readiness improvements helped demonstrate how different software components work together in a real application.

The internship experience was also valuable from a professional perspective. Working with a fixed project scope, maintaining version control, testing features individually and as an integrated system, debugging unexpected issues, and documenting the completed application provided a practical understanding of the software development process beyond classroom-based programming.

Completing Servanta has therefore provided me with both a substantial full-stack project and a stronger understanding of how software engineering principles, modern web technologies, security practices, database design, and systematic testing come together to build a complete application. The experience has strengthened my problem-solving and development skills and provided a useful foundation for my future work in software engineering and full-stack development.

14 REFERENCES

- [1] Meta Platforms, Inc., "React Documentation." [Online]. Available: <https://react.dev/>
- [2] Microsoft, "TypeScript Documentation." [Online]. Available: <https://www.typescriptlang.org/docs/>
- [3] Vite, "Vite Documentation." [Online]. Available: <https://vite.dev/guide/>
- [4] Tailwind Labs, "Tailwind CSS Documentation." [Online]. Available: <https://tailwindcss.com/docs>
- [5] OpenJS Foundation, "Node.js Documentation." [Online]. Available: <https://nodejs.org/docs/latest/api/>
- [6] OpenJS Foundation, "Express.js Documentation." [Online]. Available: <https://expressjs.com/>
- [7] PostgreSQL Global Development Group, "PostgreSQL Documentation." [Online]. Available: <https://www.postgresql.org/docs/>
- [8] Auth0, "JSON Web Tokens (JWT)." [Online]. Available: <https://jwt.io/introduction>
- [9] npm, Inc., "npm Documentation." [Online]. Available: <https://docs.npmjs.com/>
- [10] Razorpay, "Razorpay Documentation." [Online]. Available: <https://razorpay.com/docs/>
- [11] Git, "Git Documentation." [Online]. Available: <https://git-scm.com/doc>
- [12] GitHub, Inc., "GitHub Documentation." [Online]. Available: <https://docs.github.com/>
- [13] Mozilla Developer Network, "JavaScript Guide." [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide>
- [14] Mozilla Developer Network, "HTTP Documentation." [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTTP>
- [15] UniConverge Technologies Pvt Ltd, "UniConverge Technologies." [Online]. Available: <https://www.uconverge.com/>
- [16] upskill Campus, "upskill Campus." [Online]. Available: <https://www.upskillcampus.com/>

15 APPENDIX

15.1 Project Folder Structure

The folder structure below represents the overall organisation of the Servanta project, with separate directories for the React frontend and Node.js/Express backend. The structure separates user interface components, backend API functionality, database-related logic, configuration, and supporting project files.

SERVANTA/

└─ frontend/	(React.js + TypeScript frontend)
└─┬─ src/	
└─┬─┬─ components/	(reusable UI components)
└─┬─┬─ pages/	(application pages)
└─┬─┬─ layouts/	(role-based layouts)
└─┬─┬─ services/	(API communication)
└─┬─┬─ hooks/	(reusable React hooks)
└─┬─┬─ assets/	(frontend assets)
└─┬─ public/	(public frontend files)
└─┬─ package.json	
└─┬─ vite.config.ts	
└─ backend/	(Node.js + Express backend)
└─┬─ src/	
└─┬─┬─ controllers/	(business logic)
└─┬─┬─ routes/	(REST API routes)
└─┬─┬─ middleware/	(authentication and security)
└─┬─┬─ services/	(application services)
└─┬─┬─ models/	(database models/queries)
└─┬─┬─ validators/	(input validation)
└─┬─┬─ config/	(application configuration)
└─┬─ package.json	
└─┬─ tsconfig.json	
└─ database/	(database scripts and migrations)
└─ .env.example	(environment variable template)
└─ .gitignore	(ignored files and folders)
└─ docker-compose.yml	(container configuration)
└─ README.md	(project documentation)

The project structure was designed to keep the frontend, backend, and database-related components logically separated while allowing them to communicate through RESTful APIs. This organisation improves maintainability, simplifies debugging, and provides a suitable foundation for further development and deployment of Servanta.

15.2 Database Tables

Table	Key Fields	Related To
User	i, id, name, email, password_hash, role, created_at	Merchant, Customer (one-to-one)
Merchant	id, user_id, business_name, description, category, created_at	User, Service Listing, Booking
Customer	id, user_id, phone, address, created_at	User, Booking, Review, Wishlist
ServiceCategory	id, name, description, created_at	Service Listing
ServiceListing	id, merchant_id, category_id, title, description, price, status, created_at	Merchant, Booking
Booking	id, customer_id, service_id, booking_date, status, created_at	Customer, Service Listing, Payment, Review
Payment	id, booking_id, amount, status, payment_method, transaction_id, created_at	Booking
Review	id, customer_id, service_id, booking_id, rating, comment, reply, created_at	Booking, Customer, Service Listing
Wishlist	id, customer_id, service_id, created_at	Customer, Service Listing
Notification	id, user_id, type, message, is_read, created_at	User
Refund	id, payment_id, booking_id, amount, status, reason, created_at	Payment, Booking
AuditLog	id, user_id, action, entity, entity_id, created_at	User

